ALA-TOO INTERNATIONAL UNIVERSITY

ALGORITHMIZATION AND PROGRAMMING

PART II

Spring Semester 2025-2026

***

Data dominates. If you've chosen the right data structures and organized things well, the algorithms will almost always be self-evident.

— Rob Pike

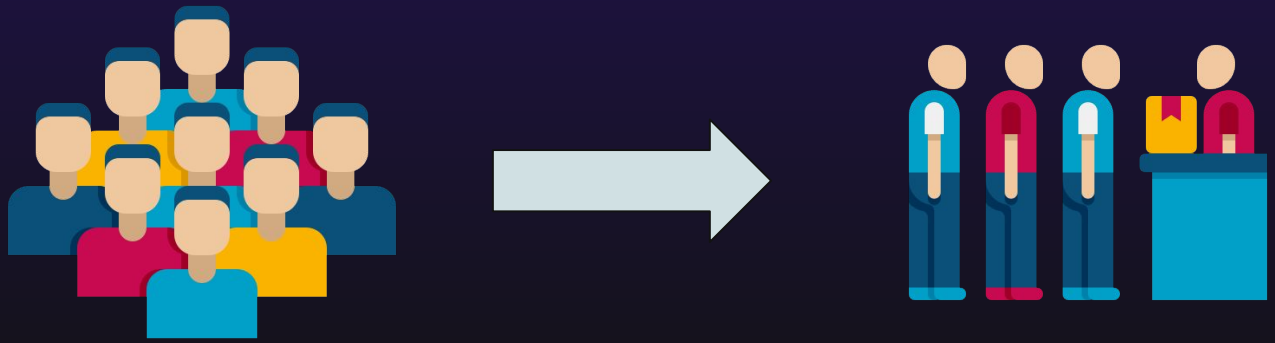
SPEED WITHOUT MAGIC

WHY THIS WEEK EXISTS

Think about a school cafeteria line. If everyone is pushing in the middle, even strong teachers can’t make it fast.

But if the line has one entrance and one exit, it becomes smooth

Data structures do the same thing: they create rules so work becomes predictable



WHAT WE BUILD TODAY

WHAT WE BUILD TODAY

We will understand and implement:

- Linked List
- Stack
- Queue
- Deque

Not as names, but as behaviors

TWO TYPES OF STORAGE

SHELF VS CHAIN

Array is like eggs in a carton. Everything sits in fixed positions next to each other: slot 0, slot 1, slot 2

Linked list is like a treasure hunt. Each clue tells you where the next clue is. You can continue, but you can’t jump instantly to the 10th clue without walking through the first 9

JAVA MEMORY SIMPLE VIEW

REFERENCES ARE ARROWS

In Java a **node** is an object on the **heap**

A reference like **head** is an arrow pointing to the first object.

If you drop the first key in a keychain, the rest of the keys are still there, but you can’t reach them.

That’s what “losing head” means.

ARRAYS VS LINKED LISTS

FAST ACCESS VS FLEXIBLE MOVE

Arrays give fast access by index: **a[7]** is immediate.
Linked list access by **index** is slow because you must walk.

Linked lists are good when you mostly add/remove at ends or you are already standing at the correct node.

RULE FOR CHOOSING

OPERATIONS FIRST ALWAYS

Before choosing a structure ask one question:

What operation happens the most?

If most actions are **get by index**, arrays usually win.
If most actions are **add/remove at ends** a queue/deque wins.
If you need **last in first out** stack wins. This is professional thinking

LINKED LIST NODE

LINKED LIST NODE

A node contains:

- The data
- A pointer to the next node

For better understanding lets use train analogy ach wagon has passengers and a connector to the next wagon.

SINGLY LINKED LIST SHAPE

ONE WAY ROAD

It’s a one-way road:

head → node → node → null

You **CAN** move forward.

You **CANNOT** move backward unless you store extra pointers.

MICRO EXAMPLE WALK

WHY INDEX IS SLOW

List: **10 → 20 → 30 → 40**

To reach 40 you must touch:
10, then 20, then 30, then 40

So “get(3)” is not one step. It’s four steps

You can’t teleport to page 40 if the book has no page numbers and only says next page.

INSERT AT HEAD

FAST ONE MOVE

If we have: **head → 10 → 20**

Insert 5:

new.next = head

head = new

Now: **head → 5 → 10 → 20**

It’s like adding a new first person in a queue by moving the sign “Start here” to them.

DELETE HEAD

FAST ONE MOVE

head → 5 → 10 → 20

Remove head means:

head = head.next

Now:

head → 10 → 20

No shifting. No moving other nodes. Just moving the entrance.

INSERT AT TAIL

WALK OR STORE TAIL

If you only know head, to add at tail you must walk to the last node every time

If you have a chain of people and you want to shake the last person’s hand, you have to pass through everyone unless you already know who the last person is.

Solution: keep tail reference.

DELETE A VALUE

NEED PREVIOUS NODE

To delete node 20 in:

10 →20 → 30

We **cannot jump directly**. We must **find the node before it**

Because the fix is:

10.next = 30

This is why deletion needs two pointers: **prev and cur**

LINKED LIST EDGE CASES

EMPTY ONE ELEMENT

Bugs happen in these cases:

Empty list: head is null

One element: head.next is null

Deleting head

Inserting into empty list

When you test, always include those cases first.

DUMMY HEAD TRICK

REMOVE SPECIAL CASES

Dummy head is a fake first node.
It makes **delete head** **NOT** special because you always delete after dummy.

Putting a buffer car on a train so coupling and uncoupling rules stay the same.

DOUBLY LINKED LIST

TWO ARROWS PER NODE

Each node has:

next

prev

This allows moving backward.

But now every insertion/deletion updates more arrows, so more chances for bugs and more memory.

DOUBLY LIST ANALOGY

STUDENT LINE WITH FRIENDS

Imagine students standing in a line but each student knows:

who is in front

who is behind

That’s doubly linked list.
It makes stepping back easy.

STACK

PLATES ON TABLE

Stack rule: last in, first out.

Analogy: plates in a stack.
You can take the top easily. You cannot pull a plate from the middle without removing plates above.

STACK OPERATIONS

PUSH POP PEEK

Push: add to top

Pop: remove from top

Peek: look at top

A stack is restricted access

That restriction makes it useful.

STACK DEMO

UNDO BUTTON LOGIC

Undo works like a stack.

Every action you do goes on top.
Undo removes the latest action.

This is why stack is **NOT** just theory. You use it daily.

STACK USING ARRAY

TOP INDEX METHOD

We store:

array

integer top

Push:

top moves up, store value

Pop:

read value, top moves down

Array stack is fast because it stays in one block of memory.

STACK USING LINKED LIST

HEAD AS TOP

Head acts as **top**

Push:

insert at head

Pop:

remove head

No resizing, but extra memory per node and more object creation.

QUEUE

CAFETERIA LINE RULE

Queue rule: first in, first out.

Analogy: cafeteria line

The first student who enters should be served first.

Queue is about fairness.

QUEUE OPERATIONS

ENQUEUE DEQUEUE PEEK

Enqueue goes to the back.

Dequeue leaves from the front.

Peek sees the front.

Important point: **queue preserves arrival order.**

COMMON BAD QUEUE

ARRAYLIST REMOVE ZERO

If you do remove(0) in an ArrayList

COMMON BAD QUEUE

ARRAYLIST REMOVE ZERO

Everyone shifts left by one. Every time.

Analogy: If the first student leaves the line and everyone physically steps forward one position every time, the line becomes slow and chaotic.

CIRCULAR QUEUE IDEA

WRAP AROUND BUFFER

We treat an array like a circle.

When we reach the end, we wrap back to index 0.

This avoids shifting.
We only move pointers.

MICRO DEMO CIRCULAR

FRONT REAR MOVEMENT

Capacity 5

Start: front=0 rear=0 size=0

Enqueue 10 → rear=1 size=1

Enqueue 20 → rear=2 size=2

Dequeue → front=1 size=1

Enqueue 30 → rear=3 size=2

Enqueue 40 → rear=4 size=3

Enqueue 50 → rear=0 wrap size=4

This is the “aha moment” students need.

QUEUE USING LINKED LIST

HEAD OUT TAIL IN

Store:

head and tail

Enqueue:

tail.next = new, tail = new

Dequeue:

head = head.next

Bug warning, when queue becomes empty, tail must also become null.

QUEUE REAL USE

BFS AND SCHEDULING

Queue appears in:

- print spoolers
- OS scheduling
- message processing
- BFS in graphs later

Queues are everywhere in real systems.

DEQUE

TWO ENDS CONTROL

Deque allows both ends operations.

Like a line where VIPs can enter from front, normal from back.
Or like a hallway where you can enter from both doors.

DEQUE REAL EXAMPLE

SLIDING WINDOW BUFFER

Think of watching last 10 sensor values.
Every second:

new value comes

oldest value leaves

Deque is the natural “rolling window” container.

ARRAY DEQUE

FASTEST PRACTICAL TOOL

ArrayDeque uses circular buffer but supports both ends.

Why it is fast in Java:

- less memory overhead
- better CPU cache
- no node allocations

LINKED DEQUE

DOUBLY LIST MODEL

A linked deque works like a doubly linked chain.
We keep head for the front and tail for the back.
So operations like addFirst, addLast, removeFirst, removeLast are fast because we only reconnect a few links.

But each node stores next and prev, so it costs more memory than an array-based deque.

BIG O WARNING

O1 CAN STILL DIFFER

Both array deque and linked deque have O(1) operations.

But array version is usually faster in real Java because constants matter:

- allocation cost
- pointer chasing
- cache misses

This is industry reality.

PROFESSIONAL CHOICE GUIDE

WHAT TO USE IN JAVA

In real Java projects, stacks and queues are often built with ArrayDeque because it stores elements in a compact array and avoids node allocations. LinkedList is a valid queue/stack too, but it usually has higher overhead: extra pointers, more objects, more pressure on memory and CPU cache.

So the lesson is not “use this class always.”
The lesson is: justify your choice by the operations you need and the performance trade-off you accept.

CLASS ACTIVITY

PICK STRUCTURE QUICKLY

Answer structure and why:

Undo feature

Printer line

Playlist next/prev

Rolling last 10 values

Insert at beginning repeatedly

Explain using operations.

CLASS ACTIVITY

UNDO FEATURE

STRUCTURE: STACK

WHY (OPERATIONS): Undo always removes the last action you did

Operations are: push(action) when you do something, pop() when you undo

That is exactly LIFO (last in, first out).

CLASS ACTIVITY

PRINTER LINE

STRUCTURE: QUEUE

WHY (OPERATIONS): The first document sent should be printed first.

Operations are: enqueue(job) when someone prints, dequeue() when printer takes next job.

That is exactly FIFO (first in, first out).

CLASS ACTIVITY

PLAYLIST NEXT/PREV

STRUCTURE: DOUBLY LINKED LIST
WHY (OPERATIONS): You need to move forward and backward quickly.

Operations are: next() and prev() many times.

A doubly linked list supports both directions naturally because every node has next and prev.

CLASS ACTIVITY

ROLLING LAST 10 VALUES

STRUCTURE: DEQUE (or CIRCULAR ARRAY BUFFER)
WHY (OPERATIONS): Every new value arrives at the end, and the oldest value leaves from the front.

Operations are: addLast(newValue) and removeFirst() repeatedly.

That is a deque behavior. In practice, ArrayDeque or a circular array is perfect.

CLASS ACTIVITY

INSERT AT BEGINNING REPEATEDLY

STRUCTURE: LINKED LIST (insert at head)
WHY (OPERATIONS): If you insert at the beginning in an array, every insert shifts all elements right (slow).

In a linked list, insert at head is just:
new.next = head; head = new

So the dominant operation (addFirst) becomes very cheap.

LAB IN CLASS

BUILD TWO IMPLEMENTATIONS

Implement:

stack using array

queue using circular array

You test:

empty pop

empty deque

wrap-around

capacity full behavior

These are real engineering tests, not only “happy path”.

HOMEWORK T008

DEQUE SERVICE SIMULATOR

T008: implement deque with circular array

Hardcoded commands in main:

addVIP(5)

addNormal(1)

addVIP(9)

serveFront

serveBack

serveFront

Print the served order.

This forces correct pointer movement and wrap-around.

WHAT IF SCENARIO

WRONG CHOICE DESTROYS SPEED

What if you build a queue using ArrayList remove(0) for 1,000,000 items
It will work but become painfully slow.

What if you choose LinkedList and then do many get(i) operations
Each get becomes a walk.

The lesson is simple:
Correct output is not enough
The structure must fit the job.

TAKE AWAY

OPERATIONS DEFINE STRUCTURE

- Arrays are fast when you need index access and fast scanning.
- Linked lists are flexible when you need fast changes at the front or when you already have the node reference, but they are slower to reach a position.
- Stack is for “last action first” problems like undo and recursion thinking.
- Queue is for “first comes first served” problems like scheduling and BFS later.
- Deque is for “both ends matter” problems like rolling windows and buffering

NEXT WEEK PREVIEW

HASHING FAST LOOKUPS

Hashing idea (turn key into index)

Hash tables and why average lookup is close to O(1)

Collisions and how real systems handle them

HashMap behavior and why performance depends on hash quality